Classification - Key Concepts

April 1, 2024

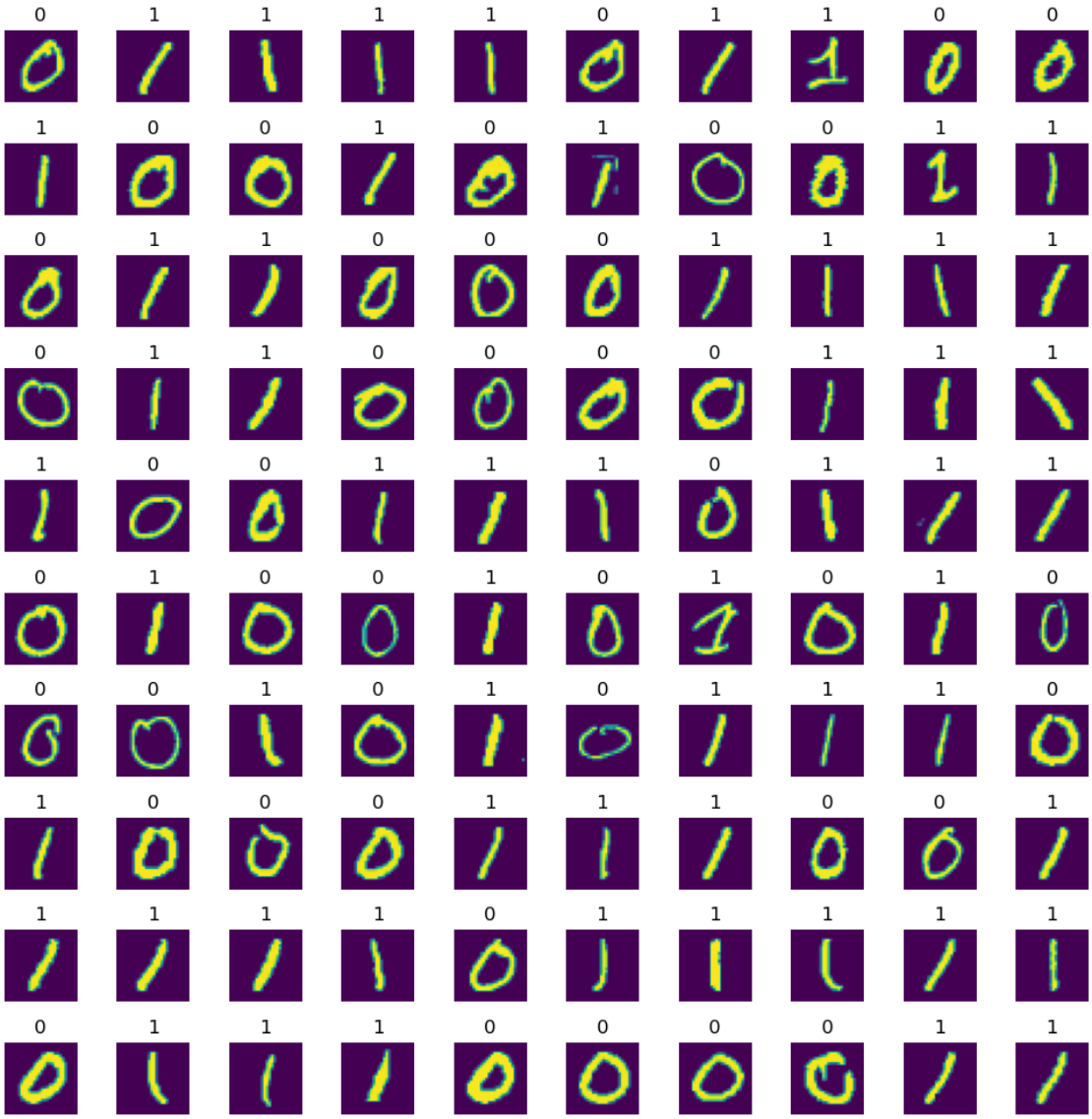
1 Classification

1.1 Explore the dataset

[1]:
```
import matplotlib.pyplot as plt
import numpy as np

lib_train = np.load('Binary_Train.npz')
X_3D, Y = lib_train['X'], lib_train['Y']

fig = plt.figure(figsize=(10, 10))
for i in range(100):
    fig.add_subplot(10, 10, i + 1)
    plt.imshow(X_3D[i])
    plt.title(Y[0, i])
    plt.axis('off')
plt.tight_layout()
plt.show()
```



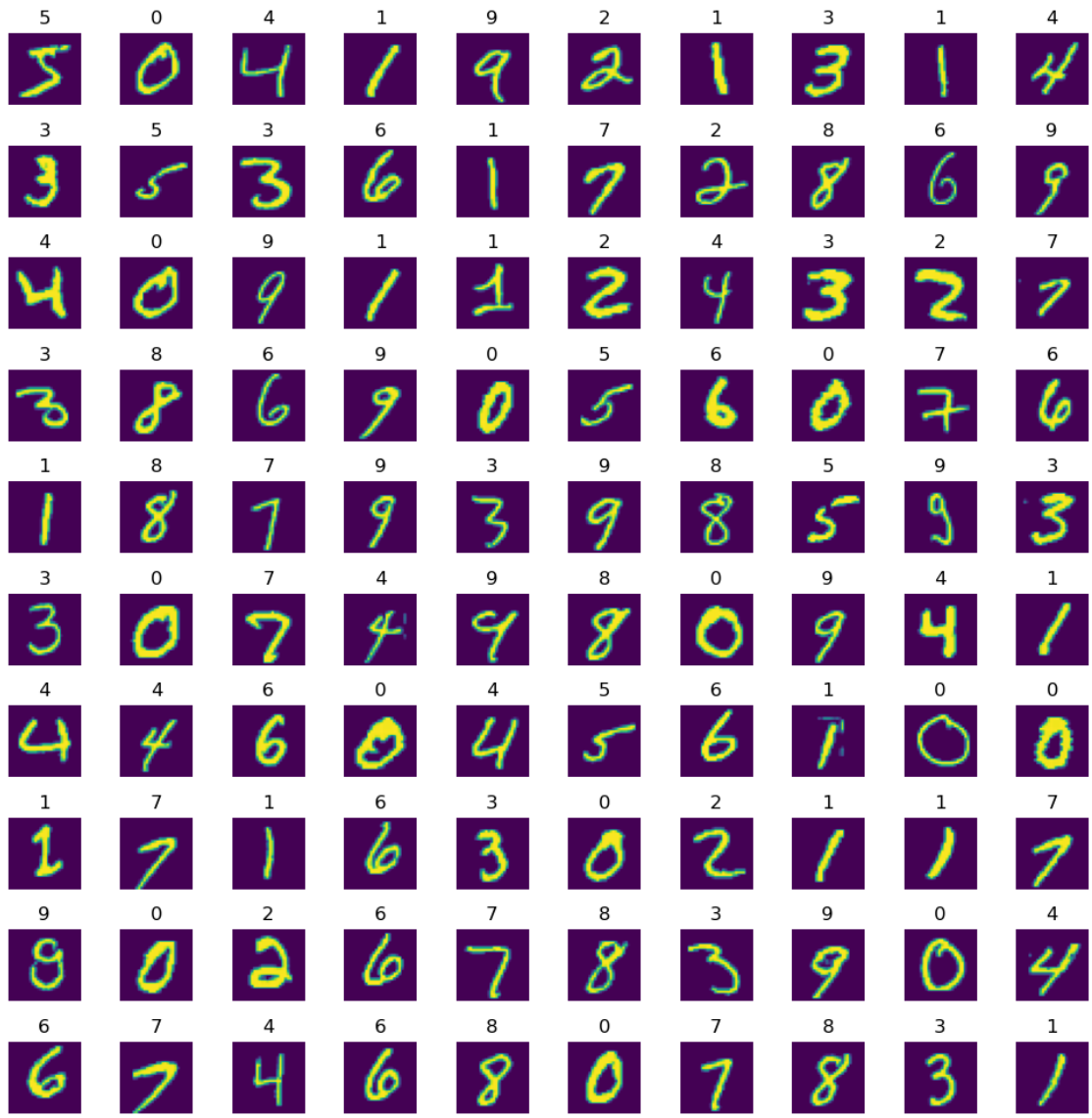
[2]:
```
import matplotlib.pyplot as plt
import numpy as np

lib = np.load('MNIST.npz')
print(lib.files)
X_train_3D = lib['X_train']
Y_train = lib['Y_train']

fig = plt.figure(figsize=(10, 10))
for i in range(100):
    fig.add_subplot(10, 10, i + 1)
    plt.imshow(X_train_3D[i])
```

```
    plt.title(str(Y_train[i]))
    plt.axis('off')
plt.tight_layout()
plt.show()
```

```
['Y_test', 'X_test', 'X_train', 'Y_train', 'X_valid', 'Y_valid']
```



1.2 Tasks

1.2.1 0. Neuron

Write a class `Neuron` that defines a single neuron performing binary classification:

- class constructor: `def __init__(self, nx)`:

  - `nx` is the number of input features to the neuron
    * If `nx` is not an integer, raise a `TypeError` with the exception: `nx must be an integer`
    * If `nx` is less than 1, raise a ValueError with the exception: `nx must be a positive integer`
  - All exceptions should be raised in the order listed above
- Public instance attributes:
  - `W`: The weights vector for the neuron. Upon instantiation, it should be initialized using a random normal distribution.
  - `b`: The bias for the neuron. Upon instantiation, it should be initialized to `0`.
  - `A`: The activated output of the neuron (prediction). Upon instantiation, it should be initialized to `0`.

1.2.2 Task 1. Privatize Neuron

Write a class `Neuron` that defines a single neuron performing binary classification (Based on `0-neuron.py`):

- class constructor: `def __init__(self, nx)`:
  - `nx` is the number of input features to the neuron
    * If `nx` is not an integer, raise a `TypeError` with the exception: `nx must be an integer`
    * If `nx` is less than 1, raise a ValueError with the exception: `nx must be a positive integer`
  - All exceptions should be raised in the order listed above
- **Private** instance attributes:
  - `__W`: The weights vector for the neuron. Upon instantiation, it should be initialized using a random normal distribution.
  - `__b`: The bias for the neuron. Upon instantiation, it should be initialized to `0`.
  - `__A`: The activated output of the neuron (prediction). Upon instantiation, it should be initialized to `0`.
  - **Each private attribute should have a corresponding getter function (no setter function).**

1.2.3 Key Concepts: Weights, Bias, Activation Function

Weights Weights in neural networks are parameters associated with the connections between neurons. They determine the strength of influence that each input feature has on the neuron's output. The weights are adjusted during training to minimize the difference between predicted and actual outputs.

Bias Bias in neural networks is an additional parameter added to each neuron. It allows the neuron to output nonzero values even when all input features are zero, providing flexibility to the model. The bias term is also learned during training and helps shift the activation function, influencing the output of the neuron.

Activation function Activation functions in neural networks introduce nonlinearity to the model by transforming the weighted sum of inputs and bias into the output of the neuron. They determine

whether the neuron should be "activated" (produce a high output) or not (produce a low output) based on the input.

1.2.4 Key Concepts: Forward Propagation

The primary aim of forward propagation in a neural network is to process input data through the network's layers, ultimately producing predictions or outputs.

More practically, during forward propagation we **compute the activation of each neuron in the network**.

This involves - calculating a weighted sum of the input data along with a bias term for each neuron - applying an activation function to produce the neuron's output.

The computed activations are passed forward through the network to generate predictions or outputs.

The sigmoid function is a commonly used activation function in neural networks. It is a mathematical function that maps any input value to a value between 0 and 1. The formula for the sigmoid function is:

$$sigmoid(z) = \frac{1}{1 + e^{-z}}$$

In the `forward_prop` method, `z` is calculated as the dot product of the weights `self.W` and input data `X`, plus the bias `self.b`. The bias is a scalar value added to each neuron's weighted inputs, which helps to shift the activation function to the left or right. Mathematically, this can be represented as:

$$z = W \cdot X + b$$

The forward_prop method returns the private attribute `self.__A`, which represents the activation of the neuron. By updating `self.__A` in the `forward_prop` method, we are able to pass the activation value to subsequent layers in the neural network during the forward propagation process.

1.2.5 Case: Neural Network

Input Data

- `x` input data matrix with shape `(nx, m)`, where `nx` is the number of input features, and `m` is the number of examples.

Activation Function:

- **Sigmoid Function**: The sigmoid function is applied to each neuron's output to introduce non-linearity into the network. It squashes the output of each neuron to a range between 0 and 1, representing probabilities.

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Intermediate Steps

Calculate Z1

- $Z^{[1]} = W^{[1]} \cdot X + b^{[1]}$

- Matrix multiplication of the weights $W^{[1]}$ and input data `X`, adding the bias term `b`

Apply Activation Function to Get A1:

- $A^{[1]} = \sigma(Z^{[1]})$
- Apply the sigmoid function to the result of the first step to get the activation of the first layer

Calculate Z2

- $Z^{[2]} = W^{[2]} \cdot A^{[1]} + b^{[2]}$
- Matrix multiplication of the weights $W^{[2]}$ and activation $A^{[1]}$ from the first layer, adding the bias term $b^{[2]}$

Apply Activation Function to Get A2

- $A^{[2]} = \sigma(Z^{[2]})$
- Apply the sigmoid function to the result of the second step to get the final output of the neural network

Output

- Returns the updated private attributes `__A1` and `__A2`, representing the activations of the first and second layers, respectively.

1.2.6 Key Concept: Cost Function

In logistic regression, the goal is to classify input data into one of two categories (0 or 1). The output of the logistic regression model is a probability value between 0 and 1, which represents the likelihood that the input belongs to the positive class (1).

To train the logistic regression model, we need to define a cost function that measures the error between the predicted probabilities and the actual labels. The cost function is then minimized using an optimization algorithm to update the model's parameters (weights and bias) such that the predicted probabilities become closer to the actual labels.

The cost function used in logistic regression is called the binary cross-entropy loss function, which is defined as follows:

$$loss = -((Y * log(A)) + ((1 - Y) * log(1 - A)))$$

The cost function is then defined as the average of the loss function over all training examples

$$J(w, b) = (1/m) * \sum(loss(y^i, a^i)), i = 1 \rightarrow m$$

1.2.7 Key Concept: Evaluate Neural Network

To evaluate a neuron model, we follow a simple process. First, we use the model to predict outcomes for a given set of input data. Then, we compare these predictions with the actual outcomes to assess how accurate the model is. Finally, we compute a measure of how well the model performed, typically referred to as the cost or loss.

In our case

- We first compute the activations of the neuron for the given input data X using the forward_prop method.
- Then, we calculate the cost of the model's predictions by comparing the computed activations A with the actual labels Y using the cost method.
- Finally, we make predictions based on the activations:
  - if an activation is greater than or equal to 0.5, we predict a positive class (1),
  - otherwise, we predict a negative class (0).

The function returns both the predictions and the computed cost, providing insights into the model's performance.

1.2.8 Key Concept: Gradient Descent

The goal of the gradient descent algorithm is to minimize the loss function and find the optimal parameters (weights and bias) for a given neural network. It does this by iteratively updating the parameters in the direction of the steepest decrease of the loss function.

Key steps of the gradient descent algorithm:

1. Calculate the derivative of the loss function with respect to the activated output (`dz`), representing the error in the predictions.
2. Compute the derivatives of the loss function with respect to the weights (`d__W`) and bias (`d__b`), indicating how much each parameter contributes to the error.
3. Update the weights and bias using one step of gradient descent, adjusting them in the opposite direction of the gradient multiplied by the learning rate (`alpha`).
4. Repeat these steps iteratively until convergence, gradually minimizing the loss function and improving the model's performance.

1.2.9 Key Concept: Train a Neural Network

The main goal of training a neural network is to iteratively adjust its parameters (weights and bias) to minimize the prediction error and improve its performance.

The training process involves repeatedly performing forward propagation to compute predictions, followed by gradient descent to update the parameters based on the prediction error.

This iterative process continues for a specified number of iterations, gradually refining the model's predictions. Finally, the training concludes with an evaluation of the trained model's performance on the training data, providing insights into its accuracy and effectiveness.

Extra: He Initialization Method He initialization is a technique used to initialize the weights of a neural network in a way that helps address the vanishing or exploding gradient problem during training.

This initialization method scales the initial weights based on the number of input and output units of each layer, leading to more stable and efficient training.

Motivation

- Traditional weight initialization methods (like random or Xavier initialization) may not be optimal for networks using ReLU activation functions.
- ReLU can cause activations to become zero for negative inputs, leading to "dead" neurons if initialized weights are too large or too small.

He Initialization:

- Named after the first author of the paper that introduced it, Kaiming He.
- Scales the initialization of weights based on the number of input units.
- Helps prevent the gradients from vanishing or exploding during training.

Formula:

$$W^{[l]} = random \cdot \sqrt{\frac{2}{n_{in}}}$$

- $W^{[l]}$ weight matrix for layer `l`
- `random`:randomly initialized weights from a standard normal distribution.
- n_{in} Number of input units to the layer `l`

1.2.10 Key Concept: One Hot Encoding

What is one hot encoding One hot encoding is a method used to **represent categorical data, like classes or labels, as binary vectors**, where each category is **uniquely identified by a vector containing all zeros except for one element, which is set to one**.

Why is it important This technique is crucial in machine learning and data analysis because it enables algorithms to efficiently work with categorical data by converting them into a format that is understandable by mathematical models.

By using one hot encoding, we transform qualitative data into a numerical format that can be processed by various machine learning algorithms, allowing for more accurate predictions and classifications in tasks such as image recognition, natural language processing, and recommendation systems.

An example In the MNIST dataset, each image represents a handwritten digit from `0` to `9`. One hot encoding is a way to represent these digits as vectors of numbers, where each digit is uniquely identified by a vector containing all zeros except for one element, which is set to one.

For example, let's take the digit 9. In traditional representation, it's just the number `9`. But with one hot encoding, we represent it as a vector like this: `[0, 0, 0, 0, 0, 0, 0, 0, 0,]`. Here, the "`1`" is in the eighth position, indicating that this is the digit `9`.

Similarly, for the digit `7`, its one hot encoded representation would be: `[0, 0, 0, 0, 0, 0, 0, 1, 0, 0]`. Here, the "`1`" is in the eighth position, indicating the digit `7`.

1.2.11 Key Concept: Pickle

Why the need to save machine learning models The primary goal of saving a model in machine learning, such as a neural network, through pickle is to **preserve its trained state and architecture for future use**, enabling easy retrieval and reuse without the need for retraining. This process is vital for various applications, as indicated by the following use cases

- **Efficient Deployment**: Saving a trained model allows for easy deployment in production environments, enabling real-time predictions and integration into software applications without the computational overhead of retraining.
- **Reproducibility**: By saving the model, researchers and practitioners can reproduce experiments and results consistently, ensuring the reliability and verifiability of machine learning workflows over time.
- **Scalability**: Saved models facilitate scalability by enabling distributed processing and parallel computing, allowing organizations to leverage large-scale computational resources efficiently for handling complex and high-volume datasets.

How to save and reuse machine learning models through pickle Pickle is a **Python module used for serializing and deserializing Python objects**. It allows objects to be converted into a byte stream, which can then be stored in a file or transmitted over a network. Pickle works by recursively traversing the object's hierarchy and converting it into a byte stream representation. When deserializing, it reconstructs the original object from the byte stream. Key methods in the pickle module include:

- `pickle.dump(object, file)`: Serializes the object and writes the byte stream representation to a file.
- `pickle.load(file)`: Deserializes the byte stream from a file and reconstructs the original object.
- `pickle.dumps(object)`: Serializes the object and returns the byte stream representation as a string.
- `pickle.loads(bytes)`: Deserializes the byte stream from a string and reconstructs the original object.

1.2.12 Happy coding